FORMULA 1 · DATA WAREHOUSE DESIGN

F1 Star Schema Architecture

A production-grade Medallion + Star Schema design for the F1 World Championship dataset. Covers PySpark ingestion from S3, dbt transformation layers, cost-optimised warehouse tables, pre-aggregated data marts, and an AI-queryable semantic layer for MCP-powered insights.

5	7	6	9	1
SCHEMA LAYERS	DIM TABLES	FACT TABLES	DATA MARTS	AI SEMANTIC LAYER

01 Medallion Architecture

S3 → PySpark → dbt → Warehouse → Marts → MCP

BRONZE · S3 RAW

Raw Zone

Kaggle CSVs + live race feeds.
Append-only. No transforms.

circuits.csv drivers.csv races.csv results.csv qualifying.csv pit_stops.csv lap_times.csv

standings.csv

SILVER · STAGING

PySpark + dbt stg_

Cast types, strip nulls,
deduplicate, add audit cols.

stg_drivers stg_races stg_results stg_qualifying stg_pit_stops stg_lap_times

GOLD · CORE DWH

Star Schema

Conformed dims + immutable facts.
SCD2 where needed.

dim_driver dim_circuit dim_constructor dim_date fact_race_result fact_qualifying

fact_pit_stop | fact_lap_time

PLATINUM · MARTS

Data Marts

Pre-aggregated for dashboard.
Materialised views or tables.

mart_driver_career | mart_constructor | mart_season_summary | mart_circuit_stats | mart_race_summary | mart_head_to_head | mart_records

AI · MCP LAYER

Semantic + AI

LLM-queryable context tables,
semantic labels, NL interface.

semantic_catalog | mart_ai_context | mcp_query_log | nl_prompt_store

TECHNOLOGY STACK

- **Storage:** AWS S3 (Bronze/Silver raw files) · Delta Lake or Iceberg format on Parquet
- **Processing:** PySpark on EMR / Databricks for initial load & incremental merges
- **Transformation:** dbt Core / Cloud — staging → core → marts, full lineage graph
- **Warehouse:** Snowflake / BigQuery / Redshift (star schema in Gold layer)
- **Orchestration:** Apache Airflow or Prefect — DAG triggers on S3 event after each race
- **AI / MCP:** Anthropic Claude + MCP server reading semantic tables for natural-language insights

02

Design Principles

Decisions that drive every table definition below

COST REDUCTION STRATEGY

- **Partition by race_year + round:** Queries for a single season scan only that partition — not 75 years of data.
- **Cluster fact tables** by driver_key and constructor_key — most dashboard queries filter on one of these.
- **Materialise marts** as physical tables, not views — the dashboard hits pre-computed numbers, never re-aggregating 1M+ lap rows.

- **fact_lap_time is the largest table** (billions of rows) — partition by year and cluster by race_key. Keep a separate `fact_lap_summary` mart for typical queries.
- **Cold storage** for raw S3 files: S3 Glacier after 30 days; only Gold/Platinum tables stay in the hot warehouse.

📐 STAR SCHEMA CONVENTIONS

- **Surrogate keys** on all dims (BIGINT auto-increment). Never join on business keys in prod queries.
- **Conformed dimensions:** dim_driver is shared across facts. One source of truth.
- **SCD Type 2** on dim_constructor (team name/nationality can change — see Benetton→Renault→Lotus→Renault).
- **Grain is explicit:** Every fact table has a grain comment in DDL and in the dbt model YAML.
- **No NULLs in FK columns** — use a "Unknown" surrogate key (-1) for late-arriving dims.
- **Immutable facts:** Historical race results never change. Only standings facts may be overwritten.

🔄 INCREMENTAL LOAD PATTERN

- After each race: S3 drop triggers Airflow DAG → PySpark reads new CSVs → MERGE into staging tables (upsert on natural key).
- dbt runs in `--select tag:incremental` mode — only rebuilds models downstream of changed source tables.
- Fact tables use dbt `incremental` materialisation with `unique_key = result_id`. New rows append; corrected rows merge.
- Marts rebuild fully after each race batch (cheap — they're aggregations of bounded-size data).

🤖 MCP / AI DESIGN CONSTRAINTS

- Every mart table has a **human-readable description** in the `semantic_catalog` table.
- The `mart_ai_context` wide table is a **single-row-per-driver** denormalised view — designed to fit in an LLM context window for "tell me about Hamilton" type queries.
- A `nl_prompt_store` table maps common question patterns to pre-built SQL, reducing AI hallucination on schema details.
- All column names are **fully spelled out** (no abbreviations) so the MCP layer can describe them without a separate lookup.

03 Staging Layer — stg_

Silver layer. PySpark read from S3 → dbt staging models. Type casting, null handling, deduplication.

```sql
-- ────────────────────────────────────────────────────────────
-- stg_drivers  (source: drivers.csv, updated_drivers feed)
-- ────────────────────────────────────────────────────────────
CREATE OR REPLACE TABLE staging.stg_drivers (
    driver_id           INT         NOT NULL,          -- Kaggle source PK
    driver_ref          VARCHAR(50) NOT NULL,
    number              INT,
    code                CHAR(3),                -- e.g. HAM, VER
    forename            VARCHAR(100)NOT NULL,
    surname             VARCHAR(100)NOT NULL,
    date_of_birth       DATE,
    nationality         VARCHAR(60),
    wikipedia_url       TEXT,
    _source_file        VARCHAR(200),           -- audit: s3 path
    _loaded_at          TIMESTAMP  DEFAULT CURRENT_TIMESTAMP,
    _is_current         BOOLEAN    DEFAULT TRUE
);

-- ────────────────────────────────────────────────────────────
-- stg_races  (source: races.csv + post-race feed)
-- ────────────────────────────────────────────────────────────
CREATE OR REPLACE TABLE staging.stg_races (
    race_id             INT         NOT NULL,
    year                SMALLINT    NOT NULL,
    round               TINYINT     NOT NULL,
    circuit_id          INT         NOT NULL,
    race_name           VARCHAR(150),
    race_date           DATE,
    race_time           TIME,
    fp1_date            DATE,   fp2_date DATE,   fp3_date DATE,
    quali_date          DATE,
    sprint_date         DATE,                   -- NULL if no sprint
    wikipedia_url       TEXT,
    _loaded_at          TIMESTAMP
);

-- Similarly: stg_results, stg_qualifying, stg_pit_stops, stg_lap_times,
-- stg_driver_standings, stg_constructor_standings, stg_constructors, stg_circuits
-- All follow same pattern: source cols + _source_file + _loaded_at + _is_current
```

04

Dimension Tables

7 conformed dimensions. Surrogate keys. SCD2 on dim_constructor. Optimised for JOIN performance.

dim_driver

1 row per driver

DIM

driver_key	BIGINT	SK
driver_id	INT	NK
driver_ref	VARCHAR	
driver_code	CHAR(3)	
full_name	VARCHAR	
forename	VARCHAR	
surname	VARCHAR	
date_of_birth	DATE	
nationality	VARCHAR	
nationality_code	CHAR(3)	IDX
flag_emoji	VARCHAR	
permanent_number	INT	
is_active	BOOLEAN	
debut_year	SMALLINT	
final_year	SMALLINT	
wikipedia_url	TEXT	
dw_created_at	TIMESTAMP	
dw_updated_at	TIMESTAMP	

dim_constructor

1 row per constructor version (SCD2)

SCD2

constructor_key	BIGINT	SK
constructor_id	INT	NK
constructor_ref	VARCHAR	
constructor_name	VARCHAR	
nationality	VARCHAR	
parent_team	VARCHAR	e.g. Renault→Alpine
is_active	BOOLEAN	
debut_year	SMALLINT	

scd_valid_from	DATE	SCD2
scd_valid_to	DATE	SCD2
scd_is_current	BOOLEAN	SCD2
wikipedia_url	TEXT	

dim_circuit

1 row per circuit

DIM

circuit_key	BIGINT	SK
circuit_id	INT	NK
circuit_ref	VARCHAR	
circuit_name	VARCHAR	
location_city	VARCHAR	
country	VARCHAR	IDX
country_code	CHAR(3)	
latitude	DECIMAL(9,6)	
longitude	DECIMAL(9,6)	
altitude_metres	INT	
circuit_type	VARCHAR	street/permanent
wikipedia_url	TEXT	

dim_season

1 row per F1 season

DIM

season_year	SMALLINT	PK (natural)
total_races	TINYINT	
has_sprint_races	BOOLEAN	
points_for_win	TINYINT	era-aware
era_label	VARCHAR	e.g. Turbo, V10, Hybrid
regulation_cycle	VARCHAR	1950/1966/1983/2022...
driver_champion_id	INT	
constructor_champion_id	INT	
wikipedia_url	TEXT	

dim_date

1 row per calendar day

DIM

date_key	INT	YYYYMMDD

full_date		DATE
year		SMALLINT
quarter		TINYINT
month_number		TINYINT
month_name		VARCHAR
day_of_week		TINYINT
day_name		VARCHAR
is_race_weekend		BOOLEAN
season_round		TINYINT

dim_race_status

1 row per result status code

DIM

status_key		INT	SK
status_id		INT	NK (Kaggle)
status_label		VARCHAR	e.g. Finished, Engine
status_category		VARCHAR	Finished / DNF / DNS / DSQ
is_classified		BOOLEAN	scored points?
is_mechanical		BOOLEAN	
is_accident		BOOLEAN	

dim_race

1 row per Grand Prix event

DIM

race_key		BIGINT	SK
race_id		INT	NK
season_year		SMALLINT	FK + PARTITION
round_number		TINYINT	
circuit_key		BIGINT	FK
race_name		VARCHAR	
race_date		DATE	
race_time_utc		TIME	
is_sprint_weekend		BOOLEAN	
race_laps		TINYINT	

05 Fact Tables

6 immutable fact tables. Explicit grain. Partitioned by year for cost control.

fact_race_result

1 row per driver × race start

FACT

result_key	BIGINT	SK
race_key	BIGINT	FK
driver_key	BIGINT	FK · CLUSTER
constructor_key	BIGINT	FK · CLUSTER
circuit_key	BIGINT	FK
status_key	INT	FK
season_year	SMALLINT	PARTITION
grid_position	TINYINT	
finish_position	TINYINT	
points_scored	DECIMAL(5,2)	
laps_completed	TINYINT	
race_time_milliseconds	BIGINT	
fastest_lap_rank	TINYINT	
fastest_lap_time_ms	INT	
fastest_lap_speed_kph	DECIMAL(6,3)	
is_fastest_lap_point	BOOLEAN	
positions_gained	SMALLINT	finish-grid delta
is_winner	BOOLEAN	
is_podium	BOOLEAN	
is_points_finish	BOOLEAN	
is_sprint	BOOLEAN	
dw_inserted_at	TIMESTAMP	

fact_qualifying

1 row per driver × qualifying session

FACT

qualify_key	BIGINT	SK
race_key	BIGINT	FK
driver_key	BIGINT	FK · CLUSTER
constructor_key	BIGINT	FK
season_year	SMALLINT	PARTITION
qualify_position	TINYINT	

q1_time_milliseconds	INT	
q2_time_milliseconds	INT	
q3_time_milliseconds	INT	
best_time_milliseconds	INT	
gap_to_pole_ms	INT	derived
gap_to_pole_pct	DECIMAL(6,4)	derived
is_pole_position	BOOLEAN	
reached_q3	BOOLEAN	
reached_q2	BOOLEAN	

fact_pit_stop

1 row per pit stop event

FACT

pit_stop_key	BIGINT	SK
race_key	BIGINT	FK
driver_key	BIGINT	FK · CLUSTER
constructor_key	BIGINT	FK
season_year	SMALLINT	PARTITION
stop_number	TINYINT	
lap_number	TINYINT	
stop_time_of_day	TIME	
stop_duration_milliseconds	INT	
race_elapsed_milliseconds	BIGINT	
is_undercut_attempt	BOOLEAN	derived

fact_lap_time

1 row per driver × lap (HUGE table)

⚠ HIGH VOL

lap_key	BIGINT	SK
race_key	BIGINT	FK · CLUSTER
driver_key	BIGINT	FK · CLUSTER
season_year	SMALLINT	PARTITION
lap_number	TINYINT	
position_on_lap	TINYINT	
lap_time_milliseconds	INT	
is_pit_lap	BOOLEAN	
is_fastest_lap	BOOLEAN	
gap_to_fastest_ms	INT	derived

stint_number	TINYINT	derived from pit data

fact_driver_standing

1 row per driver × after each race

FACT

standing_key	BIGINT	SK
race_key	BIGINT	FK (snapshot after)
driver_key	BIGINT	FK · CLUSTER
constructor_key	BIGINT	FK
season_year	SMALLINT	PARTITION
championship_position	TINYINT	
cumulative_points	DECIMAL(8,2)	
wins_to_date	TINYINT	
gap_to_leader_pts	DECIMAL(8,2)	derived
is_championship_leader	BOOLEAN	
max_remaining_points	SMALLINT	for clinch analysis

fact_constructor_standing

1 row per constructor × after each race

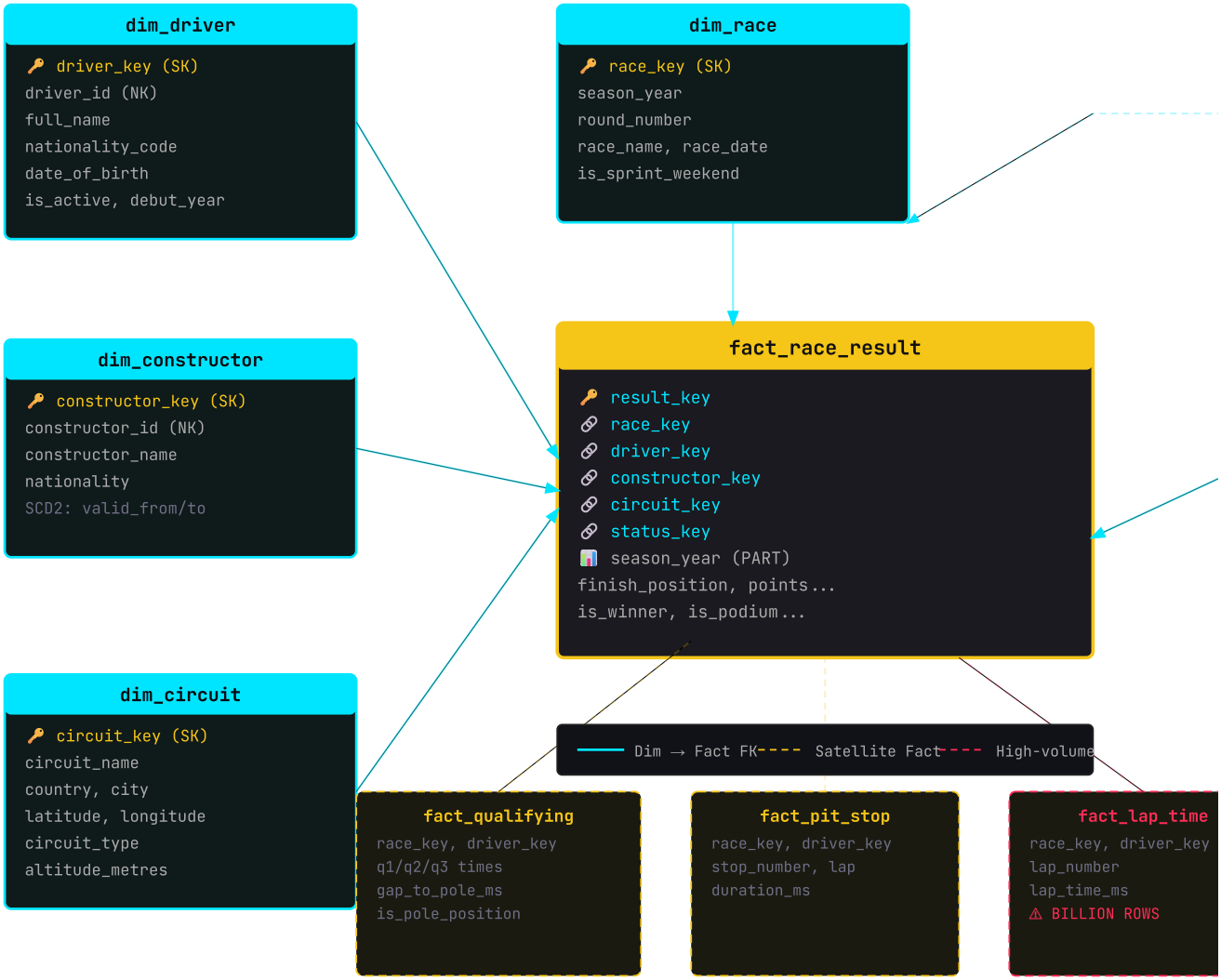
FACT

con_standing_key	BIGINT	SK
race_key	BIGINT	FK
constructor_key	BIGINT	FK · CLUSTER
season_year	SMALLINT	PARTITION
championship_position	TINYINT	
cumulative_points	DECIMAL(8,2)	
wins_to_date	TINYINT	
gap_to_leader_pts	DECIMAL(8,2)	
is_championship_leader	BOOLEAN	

06

Entity Relationship Diagram

Star schema centred on fact_race_result — the primary analytical fact



07

dbt Project Structure

Full directory tree with materialisation strategy per layer

f1_dbt/ ├── models/ | ├── staging/ -- materialised: view | schema: staging | | ├── sources.yml -- declares S3→PySpark loaded tables | | ├── stg_drivers.sql | | ├── stg_constructors.sql | | ├── stg_circuits.sql | | ├── stg_seasons.sql | | ├── stg_races.sql | | ├── stg_results.sql | | ├── stg_qualifying.sql | | ├── stg_pit_stops.sql | | ├── stg_lap_times.sql | | ├── stg_driver_standings.sql | | ├── stg_constructor_standings.sql | | └── stg_status.sql | | | ├── intermediate/ -- materialised: ephemeral (no warehouse cost) | | ├── int_drivers_enriched.sql -- adds

```
flag, nationality_code, debut_year | | ├── int_results_with_flags.sql -- adds is_winner,
is_podium, positions_gained | | ├── int_lap_times_cleaned.sql -- removes pit-out laps,
converts "1:21.546"→ms | | └── int_constructors_scd.sql -- SCD2 logic for name changes |
| | ├── core/ -- materialised: table | schema: core | | ├── dimensions/ | | | ├──
dim_driver.sql | | | ├── dim_constructor.sql | | | ├── dim_circuit.sql | | | ├──
dim_race.sql | | | ├── dim_season.sql | | | ├── dim_date.sql | | | └──
dim_race_status.sql | | └── facts/ | | ├── fact_race_result.sql -- incremental,
unique_key=result_key | | ├── fact_qualifying.sql -- incremental, unique_key=qualify_key
| | ├── fact_pit_stop.sql -- incremental, unique_key=pit_stop_key | | ├──
fact_lap_time.sql -- incremental partitioned, HUGE | | ├── fact_driver_standing.sql --
full refresh (small) | | └── fact_constructor_standing.sql | | | └── marts/ --
materialised: table | schema: marts | ├── mart_driver_career.sql | ├──
mart_driver_season.sql | ├── mart_constructor_career.sql | ├──
mart_constructor_season.sql | ├── mart_season_summary.sql | ├── mart_circuit_stats.sql |
├── mart_race_summary.sql | ├── mart_head_to_head.sql | ├── mart_records.sql | └── ai/ |
├── mart_ai_context.sql -- wide denorm for LLM context | ├── semantic_catalog.sql --
table + column descriptions | └── nl_prompt_store.sql -- NL→SQL prompt templates | ├──
macros/ | ├── generate_surrogate_key.sql | ├── ms_to_laptime.sql -- 81046 → "1:21.046" |
├── get_era_label.sql | └── flag_emoji.sql | ├── tests/ | ├── assert_no_null_fks.sql | ├──
assert_fact_grain.sql | └── assert_standing_monotonic.sql | ├── analyses/ -- ad-hoc
queries saved for reuse | ├── championship_decider_races.sql | └── head_to_head_query.sql
| └── dbt_project.yml
```

08

dbt Model SQL

Key model definitions — staging, dimension, and fact layers

```sql
-- models/staging/stg_results.sql
-- Staging model: clean raw results, cast types, add audit cols
{{ config(materialized='view', schema='staging', tags=['staging']) }}

WITH source AS (
    SELECT * FROM {{ source('raw', 'results') }}
),
renamed AS (
    SELECT
        CAST(resultId   AS INT)           AS result_id,
        CAST(raceId     AS INT)           AS race_id,
        CAST(driverId   AS INT)           AS driver_id,
        CAST(constructorId AS INT)       AS constructor_id,
```

```
        NULLIF(CAST(grid AS INT), 0)       AS grid_position,
        NULLIF(CAST(position AS INT), 0) AS finish_position,
        COALESCE(CAST(points AS DECIMAL(5,2)), 0) AS points,
        CAST(laps       AS INT)            AS laps_completed,
        CAST(statusId  AS INT)            AS status_id,
        -- parse lap time string "1:21.546" → milliseconds
        {{ ms_to_laptime('fastestLapTime') }}  AS fastest_lap_time_ms,
        NULLIF(CAST(fastestLapSpeed AS DECIMAL(6,3)), 0) AS fastest_lap_speed_kph,
        NULLIF(CAST(rank AS INT), 0)        AS fastest_lap_rank,
        'results.csv'                      AS _source_file,
        CURRENT_TIMESTAMP                  AS _loaded_at
    FROM source
    WHERE resultId IS NOT NULL
)
SELECT * FROM renamed
```

```
-- models/core/dimensions/dim_driver.sql
{{ config(materialized='table', schema='core', tags=['dim']) }}

WITH drivers AS (
    SELECT * FROM {{ ref('stg_drivers') }}
),
debut_years AS (
    SELECT
        r.driver_id,
        MIN(ra.year) AS debut_year,
        MAX(ra.year) AS final_year
    FROM {{ ref('stg_results') }} r
    JOIN  {{ ref('stg_races') }} ra ON r.race_id = ra.race_id
    GROUP BY 1
)
SELECT
    {{ generate_surrogate_key(['d.driver_id']) }}  AS driver_key,
    d.driver_id,
    d.driver_ref,
    d.code                                      AS driver_code,
    CONCAT(d.forename, ' ', d.surname)           AS full_name,
    d.forename,
    d.surname,
    d.date_of_birth,
    d.nationality,
    {{ nationality_to_code('d.nationality') }}   AS nationality_code,
    {{ flag_emoji('d.nationality') }}            AS flag_emoji,
    d.number                                    AS permanent_number,
    dy.debut_year,
    dy.final_year,
    dy.final_year ≥ YEAR(CURRENT_DATE()) - 2  AS is_active,
    d.wikipedia_url,
    CURRENT_TIMESTAMP                           AS dw_created_at,
```

```
    CURRENT_TIMESTAMP                           AS dw_updated_at
FROM drivers d
LEFT JOIN debut_years dy ON d.driver_id = dy.driver_id
```

```
-- models/core/facts/fact_race_result.sql
-- Incremental model: merge on result_key. Partition by season_year.
{{ config(
    materialized  = 'incremental',
    unique_key    = 'result_key',
    schema        = 'core',
    partition_by  = {'field': 'season_year', 'data_type': 'int'},
    cluster_by    = ['driver_key', 'constructor_key'],
    tags          = ['fact', 'incremental']
) }}

WITH results AS (
    SELECT * FROM {{ ref('stg_results') }}
    {{- if is_incremental() }}
    WHERE _loaded_at > (SELECT MAX(dw_inserted_at) FROM {{ this }})
    {{- endif }}
)
SELECT
    {{ generate_surrogate_key(['res.result_id']) }}    AS result_key,
    dr.race_key,
    dd.driver_key,
    dc.constructor_key,
    dr.circuit_key,
    ds.status_key,
    ra.year                                             AS season_year,
    res.grid_position,
    res.finish_position,
    res.points                                          AS points_scored,
    res.laps_completed,
    res.fastest_lap_time_ms,
    res.fastest_lap_speed_kph,
    res.fastest_lap_rank,
    res.fastest_lap_rank = 1                            AS is_fastest_lap_point,
    res.finish_position = 1                             AS is_winner,
    res.finish_position ≤ 3                             AS is_podium,
    res.points > 0                                      AS is_points_finish,
    res.grid_position - res.finish_position            AS positions_gained,
    FALSE                                               AS is_sprint,
    CURRENT_TIMESTAMP                                  AS dw_inserted_at
FROM results res
JOIN {{ ref('dim_race') }}        dr  ON res.race_id        = dr.race_id
JOIN {{ ref('dim_driver') }}      dd  ON res.driver_id      = dd.driver_id
JOIN {{ ref('dim_constructor') }} dc  ON res.constructor_id = dc.constructor_id
    AND dc.scd_is_current = TRUE
```

```
JOIN {{ ref('dim_race_status') }} ds  ON res.status_id      = ds.status_id
JOIN {{ ref('stg_races') }}       ra  ON res.race_id        = ra.race_id
```

09

Data Marts

9 pre-aggregated mart tables. Dashboard queries hit these — never raw facts.

mart_driver_career

1 row per driver · all-time

MART

driver_key	BIGINT
full_name	VARCHAR
nationality	VARCHAR
total_race_starts	INT
total_wins	INT
total_podiums	INT
total_poles	INT
total_fastest_laps	INT
total_points	DECIMAL
championship_titles	INT
win_rate_pct	DECIMAL(5,2)
podium_rate_pct	DECIMAL(5,2)
avg_grid_position	DECIMAL(4,2)
avg_finish_position	DECIMAL(4,2)
dnf_count	INT
career_span_years	INT
best_season_year	SMALLINT
most_wins_in_season	INT
teams_raced_for	INT
last_updated_at	TIMESTAMP

mart_driver_season

1 row per driver × season

MART

driver_season_key	BIGINT
season_year	SMALLINT

driver_key	BIGINT
full_name	VARCHAR
constructor_name	VARCHAR
races_entered	INT
season_wins	INT
season_podiums	INT
season_poles	INT
season_points	DECIMAL
final_championship_pos	INT
is_champion	BOOLEAN
pts_gap_to_champ	DECIMAL
avg_finishing_pos	DECIMAL(4,2)
best_result	INT

mart_constructor_career

1 row per constructor

MART

constructor_key	BIGINT
constructor_name	VARCHAR
nationality	VARCHAR
total_race_entries	INT
total_wins	INT
total_podiums	INT
total_poles	INT
total_points	DECIMAL
constructor_titles	INT
title_years	VARCHAR JSON array
win_rate_pct	DECIMAL(5,2)
seasons_competed	INT
active_years_range	VARCHAR
is_active	BOOLEAN
most_successful_driver	VARCHAR

mart_season_summary

1 row per season

MART

season_year	SMALLINT
driver_champion	VARCHAR

constructor_champion	VARCHAR
champion_wins	INT
champion_points	DECIMAL
total_races	INT
unique_winners	INT
unique_pole_sitters	INT
total_dnfs	INT
closest_championship_gap	DECIMAL
avg_pit_stop_duration_ms	INT
era_label	VARCHAR
has_sprint_races	BOOLEAN
championship_decided_round	INT

mart_circuit_stats

MART

1 row per circuit

circuit_key	BIGINT
circuit_name	VARCHAR
country	VARCHAR
total_races_held	INT
first_race_year	SMALLINT
last_race_year	SMALLINT
most_wins_driver	VARCHAR
most_wins_constructor	VARCHAR
lap_record_time_ms	INT
lap_record_driver	VARCHAR
lap_record_year	SMALLINT
avg_race_laps	DECIMAL(4,1)
avg_pit_stops_per_race	DECIMAL(3,1)

mart_head_to_head

MART

1 row per driver_a × driver_b × season

h2h_key	BIGINT
driver_a_key	BIGINT
driver_b_key	BIGINT
season_year	SMALLINT
races_as_teammates	INT

driver_a_quali_wins	INT	
driver_b_quali_wins	INT	
driver_a_race_wins	INT	
driver_b_race_wins	INT	
driver_a_avg_position	DECIMAL(4,2)	
driver_b_avg_position	DECIMAL(4,2)	
constructor_name	VARCHAR	
avg_quali_gap_ms	INT	

mart_records

1 row per record category

MART

record_key	INT	
record_category	VARCHAR	wins/poles/titles...
record_type	VARCHAR	driver/constructor
holder_name	VARCHAR	
record_value	DECIMAL	
record_value_formatted	VARCHAR	
set_in_year	SMALLINT	
context_note	TEXT	
prev_holder_name	VARCHAR	
prev_record_value	DECIMAL	

mart_ai_context

1 row per driver — wide denorm for LLM

AI/MCP

driver_key	BIGINT	
full_name	VARCHAR	
nationality_flag	VARCHAR	
career_summary_text	TEXT	LLM-ready prose
stats_json	JSON	all career stats
season_by_season_json	JSON	per-year breakdown
top_circuits_json	JSON	best tracks
notable_races_json	JSON	memorable moments
records_held_json	JSON	current records
context_tokens_estimate	INT	for LLM context budget
last_refreshed_at	TIMESTAMP	

10

Mart SQL

Key mart definitions — the queries your dashboard actually runs against

```
-- models/marts/mart_driver_career.sql
-- One row per driver. Full refresh after each race batch (fast — ~1000 rows).
{{ config(materialized='table', schema='marts', tags=['mart', 'driver']) }}

WITH race_stats AS (
    SELECT
        f.driver_key,
        COUNT(*)                                          AS total_race_starts,
        SUM(CASE WHEN f.is_winner      THEN 1 ELSE 0 END)  AS total_wins,
        SUM(CASE WHEN f.is_podium      THEN 1 ELSE 0 END)  AS total_podiums,
        SUM(CASE WHEN f.is_fastest_lap_point THEN 1 ELSE 0 END) AS total_fastest_laps,
        SUM(f.points_scored)                               AS total_points,
        AVG(f.grid_position)                               AS avg_grid_position,
        AVG(f.finish_position)                             AS avg_finish_position,
        SUM(CASE WHEN s.status_category = 'DNF' THEN 1 ELSE 0 END) AS dnf_count
    FROM   {{ ref('fact_race_result') }}  f
    JOIN   {{ ref('dim_race_status') }}  s  ON f.status_key = s.status_key
    WHERE  f.is_sprint = FALSE
    GROUP BY 1
),
pole_stats AS (
    SELECT
        driver_key,
        SUM(CASE WHEN is_pole_position THEN 1 ELSE 0 END) AS total_poles
    FROM   {{ ref('fact_qualifying') }}
    GROUP BY 1
),
title_count AS (
    SELECT
        driver_key,
        COUNT(*) AS championship_titles
    FROM   {{ ref('mart_season_summary') }}  ss
    JOIN   {{ ref('dim_driver') }}           dd  ON ss.driver_champion = dd.full_name
    GROUP BY 1
)
SELECT
    d.driver_key,
    d.full_name,  d.nationality,  d.flag_emoji,
    d.date_of_birth, d.debut_year, d.final_year, d.is_active,
    rs.total_race_starts,
    COALESCE(rs.total_wins, 0)          AS total_wins,
    COALESCE(rs.total_podiums, 0)       AS total_podiums,
    COALESCE(ps.total_poles, 0)         AS total_poles,
    COALESCE(rs.total_fastest_laps, 0)  AS total_fastest_laps,
```

```
    COALESCE(rs.total_points, 0)        AS total_points,
    COALESCE(tc.championship_titles, 0) AS championship_titles,
    ROUND(rs.total_wins * 100.0 / NULLIF(rs.total_race_starts, 0), 2) AS win_rate_pct,
    ROUND(rs.avg_grid_position, 2)    AS avg_grid_position,
    ROUND(rs.avg_finish_position, 2)  AS avg_finish_position,
    COALESCE(rs.dnf_count, 0)          AS dnf_count,
    CURRENT_TIMESTAMP                   AS last_updated_at
FROM {{ ref('dim_driver') }}           d
LEFT JOIN race_stats                   rs  ON d.driver_key = rs.driver_key
LEFT JOIN pole_stats                   ps  ON d.driver_key = ps.driver_key
LEFT JOIN title_count                  tc  ON d.driver_key = tc.driver_key
ORDER BY total_wins DESC
```

11

Partitioning & Cost Strategy

Every decision justified by query scan reduction

⚠️ THE GOLDEN RULE

The `fact_lap_time` table will reach ~10 billion rows by 2030. **Never let the dashboard query it directly.** All lap-time queries go through `mart_driver_season` or aggregated materialized views. Direct lap queries are only for data engineering and deep analysis use-cases.

TABLE	PARTITION COLUMN	CLUSTER / SORT KEYS	MATERIALISATION	ESTIMATED ROWS
fact_race_result	season_year (INT)	driver_key, constructor_key	Incremental table	~50K
fact_qualifying	season_year (INT)	driver_key, race_key	Incremental table	~40K
fact_pit_stop	season_year (INT)	driver_key, race_key	Incremental table	~120K
fact_lap_time	season_year (INT), then race_key	driver_key	Incremental table	>500M
fact_driver_standing	season_year (INT)	driver_key	Full refresh table	~25K

TABLE	PARTITION COLUMN	CLUSTER / SORT KEYS	MATERIALISATION	ESTIMATED ROWS
fact_constructor_standing	season_year (INT)	constructor_key	Full refresh table	~8K
mart_driver_career	None	total_wins DESC	Full refresh table	~1K
mart_driver_season	season_year	driver_key	Full refresh table	~20K
mart_season_summary	None	season_year	Full refresh table	75 rows
mart_ai_context	None	driver_key	Full refresh table	~1K

💡 BIGQUERY OPTIMISATIONS

- Use **INTEGER RANGE partitioning** on season_year (1950–2030, interval=1).
- Enable **column-level stats** on driver_key and constructor_key for partition pruning hints.
- Set **table expiry** on staging tables: 7 days. They're re-loadable from S3.
- **Flat-rate pricing** for the dashboard (BI Engine reservation) — marts are queried constantly.
- Use **materialized views** with `enable_refresh=true` for the top-5 dashboard queries.

❄️ SNOWFLAKE OPTIMISATIONS

- Use **CLUSTER BY (season_year, driver_key)** on all main fact tables.
- Suspend warehouse after **1 minute idle** — dashboard queries are sub-second on marts.
- Use **SEARCH OPTIMIZATION** on dim_driver.full_name for AI/MCP free-text lookups.
- Store mart tables as **Transient** (no fail-safe) — they rebuild from facts on every run.
- Use **Multi-cluster warehouses** only during race-day batch loads.

12

MCP / AI Semantic Layer

How Claude queries this warehouse using natural language via MCP

🤖 ARCHITECTURE: HOW THE AI LAYER WORKS

An MCP server exposes warehouse read access to Claude. The `semantic_catalog` table gives Claude plain-English descriptions of every table and column. The `nl_prompt_store` maps common question patterns to pre-validated SQL. The `mart_ai_context` wide table lets Claude answer driver questions with a single primary-key lookup — no joins needed in the LLM context.

```
-- models/marts/ai/semantic_catalog.sql
-- The MCP server reads this table to understand the schema before generating SQL.
{{ config(materialized='table', schema='marts_ai') }}

SELECT * FROM (
    VALUES
    -- (schema, table_name, column_name, data_type, description, example_values, is_filteral

    ('marts', 'mart_driver_career', 'full_name',          'VARCHAR',
     'Full name of the Formula 1 driver in "Forename Surname" format',
     'Lewis Hamilton, Max Verstappen, Ayrton Senna', TRUE),

    ('marts', 'mart_driver_career', 'championship_titles', 'INT',
     'Number of F1 World Driver Championship titles won in their career',
     '7 (Hamilton/Schumacher), 5 (Fangio), 4 (Vettel)', TRUE),

    ('marts', 'mart_driver_career', 'total_wins',          'INT',
     'Total race wins in the drivers F1 career, excluding sprint races',
     '103 (Hamilton), 91 (Schumacher), 62 (Verstappen)', TRUE),

    ('marts', 'mart_driver_career', 'win_rate_pct',        'DECIMAL',
     'Percentage of race starts that resulted in a win. Use for era-adjusted comparisons.',
     '46.2 (Fangio), 34.2 (Clark), 31.8 (Verstappen)', TRUE),

    ('marts', 'mart_season_summary', 'season_year',         'SMALLINT',
     'The F1 World Championship year, ranging from 1950 to present',
     '2024, 2023, 2021, 1994, 1976', TRUE),

    ('marts', 'mart_season_summary', 'closest_championship_gap', 'DECIMAL',
     'Points gap between champion and runner-up at season end. 0.5 points in 1984 is the clo
     '0.5 (1984), 7.5 (2021), 8 (2008)', TRUE),

    ('core', 'fact_race_result', 'season_year',          'SMALLINT',
     'PARTITION KEY. Always filter on this for cost-efficient queries.',
     '1950..2024', TRUE)

    -- ... 200+ more rows covering every relevant column
) AS t(schema_name, table_name, column_name, data_type, description, example_values, is_filt
```

```
-- models/marts/ai/nl_prompt_store.sql
-- Pre-validated SQL templates for common natural-language questions.
-- MCP server picks the closest match and fills in parameters.
{{ config(materialized='table', schema='marts_ai') }}

SELECT * FROM (
    VALUES
    -- (prompt_id, intent, example_nl_question, sql_template, required_params)

    (1, 'driver_career_stats',
     'How many wins does {driver} have?',
     'SELECT total_wins, total_podiums, total_poles, championship_titles
      FROM marts.mart_driver_career
      WHERE LOWER(full_name) LIKE LOWER(:driver_name)
      LIMIT 1',
     '{driver_name}'),

    (2, 'season_standings',
     'Who won the {year} championship?',
     'SELECT driver_champion, constructor_champion, champion_wins, champion_points,
             total_races, closest_championship_gap
      FROM marts.mart_season_summary
      WHERE season_year = :year',
     '{year}'),

    (3, 'head_to_head',
     'Compare {driver_a} vs {driver_b}',
     'SELECT a.full_name AS driver_a, a.total_wins AS a_wins, a.championship_titles AS a_tit
             b.full_name AS driver_b, b.total_wins AS b_wins, b.championship_titles AS b_tit
             a.win_rate_pct AS a_win_rate, b.win_rate_pct AS b_win_rate
      FROM marts.mart_driver_career a
      CROSS JOIN marts.mart_driver_career b
      WHERE LOWER(a.full_name) LIKE LOWER(:driver_a)
        AND LOWER(b.full_name) LIKE LOWER(:driver_b)',
     '{driver_a}, {driver_b}'),

    (4, 'circuit_history',
     'Who has the most wins at {circuit}?',
     'SELECT d.full_name, COUNT(*) AS wins_at_circuit
      FROM core.fact_race_result f
      JOIN core.dim_driver d ON f.driver_key = d.driver_key
      JOIN core.dim_race dr ON f.race_key = dr.race_key
      JOIN core.dim_circuit c ON dr.circuit_key = c.circuit_key
      WHERE LOWER(c.circuit_name) LIKE LOWER(:circuit_name) AND f.is_winner = TRUE
      GROUP BY 1 ORDER BY 2 DESC LIMIT 5',
     '{circuit_name}'),

    (5, 'constructor_dominance',
     'Which team dominated the {decade}s?',
     'SELECT c.constructor_name, COUNT(*) AS titles
```

```
        FROM marts.mart_season_summary ss
        JOIN core.dim_constructor c ON ss.constructor_champion = c.constructor_name
        WHERE ss.season_year BETWEEN :decade_start AND :decade_start + 9
        GROUP BY 1 ORDER BY 2 DESC',
       '{decade_start}')

  ) AS t(prompt_id, intent, example_nl_question, sql_template, required_params)
```

The MCP server flow:

13

Post-Race Ingestion Flow

What happens in the hours after each Grand Prix

```
## Airflow DAG: f1_post_race_pipeline
## Trigger: S3 PutObject event on s3://f1-raw/live-feed/YYYY/race_NNN/
## Target SLA: Dashboard updated within 3 hours of chequered flag

DAG: f1_post_race_pipeline
  schedule: S3 event trigger (EventBridge → SNS → Airflow API)
  concurrency: 1  # Only one race at a time

  1. s3_sensor
      ↳ Watches s3://f1-raw/live-feed/{year}/race_{round}/complete.flag
      ↳ Timeout: 6 hours post-race

  2. pyspark_ingest  [EMR Serverless / Databricks Job]
      ↳ Read new CSV files from S3 Bronze partition
      ↳ Schema validation (Great Expectations)
      ↳ MERGE into Silver staging tables (upsert on natural key)
      ↳ Write audit record to pipeline_log

  3. dbt_run_incremental
      ↳ dbt run --select tag:incremental --vars '{"race_id": NNN}'
```

```
    ↳ Builds: fact_race_result, fact_qualifying, fact_pit_stop, fact_lap_time
    ↳ Builds: fact_driver_standing, fact_constructor_standing

4. dbt_run_marts
    ↳ dbt run --select marts.*
    ↳ Full refresh of all 9 mart tables (~2 min total)

5. dbt_test
    ↳ dbt test --select marts.*
    ↳ Assert: no null FKs, standings are monotonic, grain is unique

6. invalidate_cache
    ↳ POST /api/cache/invalidate on dashboard service
    ↳ Notify: Slack #f1-data-ops channel

7. mcp_context_refresh
    ↳ Rebuild mart_ai_context for drivers with new data
    ↳ Update mcp_query_log with race metadata

Total pipeline time: ~25-40 minutes end-to-end
```

✅ DATA QUALITY GATES

- **Row count check:** New race must have 16–24 result rows (modern grid size). Pipeline fails if outside range.
- **Points validation:** Sum of points in a race must match expected total for the era (e.g., top-10 points system post-2010).
- **Referential integrity:** Every result's driver_id and constructor_id must exist in dim tables before fact load.
- **Duplicate guard:** MERGE on (race_id, driver_id) prevents double-loading the same race.
- **Freshness alert:** If staging tables haven't received new rows within 12 hours of race day, PagerDuty alert fires.

📋 SUMMARY — FULL TABLE INVENTORY

STAGING (13)	CORE DIMS (7) + FACTS (6)	MARTS (9 + 3 AI)
stg_drivers	dim_driver, dim_constructor	mart_driver_career
stg_constructors	dim_circuit, dim_race	mart_driver_season
stg_circuits, stg_seasons	dim_season, dim_date	mart_constructor_career
stg_races, stg_results	dim_race_status	mart_constructor_season
stg_qualifying	fact_race_result	mart_season_summary
stg_pit_stops, stg_lap_times	fact_qualifying	mart_circuit_stats
stg_driver_standings	fact_pit_stop, fact_lap_time	mart_race_summary
stg_constructor_standings	fact_driver_standing	mart_head_to_head

stg_status

stg_sprint_results

fact_constructor_standing

mart_records

mart_ai_context

semantic_catalog

nl_prompt_store